

Projeto UC00606



Projeto UC00606

Objetivos Gerais e Descrição do Problema	3
Requisitos Funcionais	3
UC1: Importação de Dados	3
UC2: Sistema de Exclusão	3
UC3: Extração de Padrões Base	3
UC4: Modelação e Encapsulamento de Eventos	3
UC5: Categorização de Ameaças	4
UC6: Avaliação de Risco e Polimorfismo	4
UC7: Formatação e Listagem	4
UC8: Monitorização Global de Estado	4
UC9: Agregação em Memória e Detecção de Anomalias	4
UC10: Pesquisa Forense Isolada e Ordenação	5
UC11: Interface do Utilizador e Triagem	5
UC12: Exportação de Relatórios no Encerramento	5
Requisitos Não Funcionais	6
Notas de Apoio	7

Objetivos Gerais e Descrição do Problema

O presente projeto visa a construção de uma ferramenta de linha de comandos (CLI) desenhada para ingerir, processar e categorizar dados não estruturados, transformando-os em inteligência de segurança operacional. Uma infraestrutura servidora foi alvo de múltiplos ataques, resultando num ficheiro de registo de eventos em formato de texto bruto. A missão consiste em desenvolver uma arquitetura em Python que processe este ficheiro, identifique ameaças através de padrões, aloque a informação em memória de forma encapsulada e exporte um relatório estruturado. A solução exige a aplicação rigorosa de Controlo de Versões (Git), Expressões Regulares (Regex) e Programação Orientada a Objetos (OOP).

Requisitos Funcionais

UC1: Importação de Dados

O sistema deve iniciar a sua execução solicitando ao utilizador o nome do ficheiro de registos principal a analisar. É exigido que a aplicação suporte proativamente falhas na localização do ficheiro, intercetando anomalias do sistema operativo e apresentando alertas claros sem interromper a execução do programa.

UC2: Sistema de Exclusão

Antes de instanciar os registos, o sistema deve suportar o carregamento de um ficheiro secundário contendo endereços IP validados como seguros (tráfego interno). Estes endereços têm de ser carregados para memória utilizando obrigatoriamente a estrutura de dados Conjunto (*Set*). Se o IP de um evento constar nesta lista, o sistema deve descartar a linha silenciosamente.

UC3: Extração de Padrões Base

A transformação das linhas de texto bruto em dados processáveis tem de ser executada exclusivamente através da biblioteca *re* (Expressões Regulares). O motor de extração deve ser capaz de isolar com rigor a data e hora da ocorrência, o endereço IP de origem e a descrição do evento, descartando linhas corrompidas ou mal formatadas.

UC4: Modelação e Encapsulamento de Eventos

A arquitetura exige a criação de uma classe base genérica designada *LogEvent*. A memória da classe tem de ser estritamente privada. A injeção de dados durante a instanciação deve ser sempre canalizada por métodos de controlo clássicos (*setters*) ou propriedades decoradas (*@property*), rejeitando liminarmente a criação de instâncias com endereços IP estruturalmente inválidos através do levantamento de exceções lógicas.

UC5: Categorização de Ameaças

Devem ser implementadas classes derivadas específicas, tais como *AuthFailureEvent*, *SqliInjectionEvent* e *PortScanEvent*. O processo de inicialização destas classes filhas tem de utilizar o método *super()* para delegar o processamento dos dados estruturais partilhados na classe mãe.

UC6: Avaliação de Risco e Polimorfismo

O sistema tem de prever um método genérico de cálculo de risco na classe base *LogEvent*, devolvendo um valor nulo (0). Este método está sujeito a sobrescrita (*overriding*) obrigatória nas classes derivadas. O cálculo da pontuação de severidade deve ser dinâmico, obrigando a classe a aceder ao seu próprio estado encapsulado. A fórmula a aplicar obrigatoriamente é: $\text{Risco Total} = \text{Risco Base} + \text{Comprimento da Mensagem}$ (número total de caracteres da mensagem extraída). Os valores de Risco Base a utilizar são: *SqliInjectionEvent* (Base 10), *AuthFailureEvent* (Base 5) e *PortScanEvent* (Base 3).

UC7: Formatação e Listagem

A representação textual dos objetos em memória não pode depender de formatações impostas externamente. Cada classe deve reescrever os métodos especiais nativos do interpretador (*__str__* e *__repr__*) para gerar saídas perfeitamente formatadas. Adicionalmente, o sistema deve disponibilizar um mecanismo de listagem em ecrã onde o utilizador escolhe o que deseja visualizar: a listagem global de todos os eventos ou uma listagem filtrada por categoria de ameaça (ex: listar apenas ocorrências de *SqliInjectionEvent*, apenas *AuthFailureEvent* ou apenas *PortScanEvent*).

UC8: Monitorização Global de Estado

A classe base *LogEvent* tem de implementar um atributo de classe inteiro (ex: *total_threats*), inicializado a zero. Sempre que o construtor for executado com sucesso e uma nova ameaça for instanciada, este contador global partilhado deve ser incrementado, mantendo o registo absoluto de incidentes processados na sessão.

UC9: Agregação e Detecção de Anomalias

O ciclo de vida da aplicação será gerido por uma classe orquestradora (ex: *ThreatAnalyzer*) que centraliza os eventos numa lista de instâncias. O sistema tem de analisar esta lista recorrendo a Dicionários para mapear o volume de tráfego por IP. O mapeamento deve sinalizar automaticamente origens que registem 5 ou mais falhas de autenticação, categorizando-as como tentativas de força bruta.

UC10: Pesquisa Isolada e Ordenação

A ferramenta deve disponibilizar uma auditoria avançada. Ao receber um IP específico do utilizador, o motor deverá iterar sobre a memória, filtrar o histórico de incidentes associados e devolver a lista ordenada do risco mais crítico para o menos crítico. Em caso de empate na pontuação de risco, a ordenação secundária deve ser feita de forma cronológica (do evento mais recente para o mais antigo).

UC11: Interface do Utilizador

A interação decorrerá através de um menu de linha de comandos em ciclo contínuo (*while True*). A interface tem de ser resiliente a erros de inserção de dados (*try-except*), garantindo a navegação estável entre as seguintes funcionalidades:

1. Input de ficheiros (Logs e Whitelist);
2. Apresentação do total global de ameaças (UC8);
3. Listagem de eventos globais ou filtrada por tipologia (aciona UC7);
4. Detecção de anomalias agregadas (UC9);
5. Auditoria específica a IPs com ordenação de risco (UC10);
6. Sair e Exportar Relatório Final (aciona UC12).

UC12: Exportação de Relatórios

A extração dos resultados da análise para o disco rígido constitui o fim absoluto do fluxo de processamento. Quando o utilizador selecionar a opção de sair da aplicação no menu principal, o sistema deve, de forma automática e antes de terminar a execução, gerar um ficheiro de texto (ex: report.txt). O código responsável por esta operação deve garantir a criação e o fecho seguro do ficheiro, prevenindo a corrupção de dados. O relatório exportado tem de conter, obrigatoriamente e de forma legível, a seguinte informação:

- O número total absoluto de ameaças instanciadas e processadas durante a sessão (extraído do UC8).
- A lista exata de endereços IP catalogados como ataques de força bruta (extraído do UC9).
- O Top 3 dos endereços IP com maior risco global acumulado, acompanhados da respetiva pontuação de severidade.

Requisitos Não Funcionais

O código lógico (classes, métodos e variáveis) deverá ser integralmente redigido em Inglês (utilizando snake_case e PascalCase).

Exige-se a aplicação rigorosa de tipagem estrita (Type Hinting) e documentação de métodos (Docstrings).

O projeto será efetuado em grupos de 3 elementos (com exceção para um grupo de 2 elementos).

O uso de um repositório Git é obrigatório. Os grupos devem conceder acesso ao repositório ao formador. Exige-se a utilização de ramificações (branches) para separação de tarefas e a adoção da norma Conventional Commits.

Prazo e Submissão: A entrega final tem como data limite o dia 1 de julho às 23h59, devendo ser efetuada exclusivamente através da plataforma Moodle.

Formato de Entrega: A submissão consistirá num único ficheiro .zip contendo os scripts Python (pode ser feito download do repositório através do Git). Basta um elemento de cada grupo submeter o trabalho. A nomenclatura do ficheiro deve indicar o número do grupo e o primeiro e último nome de cada elemento, separados por underscores.

Exemplo: **G1_LuisVigario_NomeApelido_NomeApelido.zip**

A avaliação implica uma apresentação e defesa oral individual. A classificação do projeto está diretamente dependente da capacidade do formando em justificar as opções de código e a arquitetura adotada.

Notas de Apoio

Para auxiliar o planeamento inicial e evitar impasses no fluxo de dados, o grupo deve considerar as seguintes diretrizes estruturais:

- **Fluxo de Processamento de Dados (UC1 a UC5):** Não tentem ler o ficheiro e criar objetos no mesmo passo. O fluxo lógico correto é uma "linha de montagem":
 1. Ler uma linha de texto do ficheiro;
 2. Verificar se o IP dessa linha existe no Conjunto (Set) da *Whitelist* (UC2);
 3. Se não existir, passar a linha de texto pelo motor Regex para extrair as variáveis limpas (UC3);
 4. Utilizar as variáveis extraídas para instanciar o objeto de ataque correto (UC4 e UC5);
 5. Entregar esse objeto finalizado ao ThreadAnalyzer.
- **Seleção de Objetos (UC4 e UC5):** Como é que o sistema sabe qual a classe derivada a instanciar para uma determinada linha do ficheiro? Após a expressão regular separar o IP, a Data e a Mensagem, o vosso código deve avaliar o conteúdo textual da Mensagem através de estruturas condicionais (if / elif). Por exemplo, se a mensagem extraída (convertida para minúsculas com `.lower()` para evitar falhas) contiver palavras-chave como "failed password" ou "authentication", o sistema deve instanciar um `AuthFailureEvent`. Se a mensagem contiver "syntax error" ou "union select", instancia um `SqlInjectionEvent`. Este bloco de código atua como um centro de triagem, analisando a *string* e construindo o objeto adequado antes de o entregar ao Orquestrador.
- **Filtragem de Objetos por Tipo (UC7):** Para apresentarem apenas os eventos de um determinado tipo sem duplicar listas em memória, devem pesquisar sobre as funções embutidas do Python desenhadas para interrogar a natureza de um objeto durante a iteração (como o `isinstance()` ou a avaliação do atributo `__class__`).
- **Eficiência na Leitura de Ficheiros:** Evitem carregar o conteúdo integral de um ficheiro de registos de uma só vez para a memória RAM (utilizando o método `.read()`). Em cibersegurança, ficheiros de *logs* podem ter gigabytes de tamanho. Utilizem a iteração linha a linha (ex: `for line in file:`).
- **Ficheiros e Autonomia:** A sintaxe de abertura e fecho seguro de ficheiros de texto não faz parte do guião. O grupo deve investigar de forma autónoma a instrução `with open(...)` em Python.

- **Construção de Regex (UC3):** A extração de texto torna-se complexa se tentarem validar a linha inteira de uma só vez. Construam e testem o padrão de forma modular: construam a captura da data, depois adicionem a captura do endereço IP, e finalmente a extração da mensagem.
- **Atributos de Classe vs Instância (UC8):** Recordem que um atributo de classe é declarado diretamente no corpo da classe, fora do método `__init__`, e deve ser acedido e atualizado invocando o nome da própria classe (ex: `LogEvent.total_threats`).
- **O Conceito de Herança (UC5):** Validem sempre a relação lógica da estrutura. A Herança traduz-se numa relação "É um". Uma injeção SQL **é um** evento de registo (*log*), logo faz sentido herdar os atributos base em vez de os recriar do zero.
- **O Papel do Orquestrador e o Isolamento de Estruturas (UC9 e UC10):** A classe `ThreatAnalyzer` não é um evento de registo é o orquestrador do programa. Deve ser instanciada apenas uma vez, no início da função `main()`. Esta classe tem de conter uma lista privada (ex: `self.__events_list`) e um método para receber e guardar todos os novos objetos instanciados.
- **Agregação com Dicionários (UC9):** O objetivo desta estrutura é criar um mapeamento temporário de Chave-Valor. O código deve iterar sobre a lista principal de objetos, extrair o IP de cada objeto (Chave) e somar +1 ao Valor inteiro correspondente sempre que o encontrar. Exemplo do estado final pretendido em memória: `{"192.168.1.10": 15, "10.0.0.5": 5}`. Notem que este dicionário serve estritamente para contagem de volume; ao criá-lo, o rasto aos dados detalhados de cada evento (data, mensagem, risco) é perdido nessa estrutura.
- **Pesquisa e Ordenação (UC10):** Para resolver o UC10 (que exige a listagem detalhada e ordenada), não tentem extrair dados do dicionário do UC9. O motor deve voltar a iterar sobre a lista principal de objetos, filtrar os objetos que correspondem ao IP procurado para uma nova lista temporária, e ordenar essa lista temporária com base no valor devolvido pelo método de cálculo de risco.